

Reporte de Proyecto: Motor de Inferencia Legislativo (API REST + Prolog)

1. Introducción

Este proyecto consiste en el desarrollo de un **Motor de Inferencia Legislativo** capaz de evaluar automáticamente el cumplimiento de contratos y determinar penalizaciones basadas en reglas preestablecidas. La solución integra dos tecnologías potentes: **Node.js** para la gestión de servicios web y **Prolog** para el razonamiento lógico. Esta combinación permite que un sistema moderno pueda realizar consultas complejas sobre una base de conocimientos de manera eficiente y escalable.

2. Explicación de Paradigmas de Programación Usados

Para la resolución de este problema, se implementó un enfoque **multiparadigma**:

- **Paradigma Lógico (Prolog):** Es el corazón del proyecto. A diferencia de la programación imperativa, aquí no le decimos a la computadora *cómo* hacer las cosas, sino que definimos un conjunto de **hechos** y **reglas**. El motor de inferencia de Prolog utiliza la unificación y el backtracking para encontrar soluciones a partir de las premisas dadas en la base de conocimientos.
- **Paradigma Funcional y Asíncrono (Node.js/JavaScript):** Se utilizó para construir la interfaz de comunicación (API). JavaScript gestiona las peticiones de entrada y salida de forma no bloqueante, lo que permite que el servidor web sea ligero y rápido mientras espera las respuestas del motor lógico.

3. Arquitectura

La arquitectura del sistema sigue un modelo de **Microservicio Desacoplado**:

1. **Capa de Interfaz (API REST):** Desarrollada con **Express**. Expone un endpoint (/query) que recibe datos en formato **JSON**. Su función es actuar como puente entre el usuario y la lógica.
2. **Capa de Adaptación (Middleware):** El servidor recibe el JSON, extrae la consulta lógica y la traduce para que el motor de Prolog pueda interpretarla.
3. **Capa de Lógica (Motor Prolog):** Utilizando la librería **Tau Prolog**, se carga un archivo de base de conocimiento (.pl). Este motor procesa la consulta contra las reglas de contratos y devuelve los resultados deducidos.

4. Ejemplo de Ejecución

Para validar el funcionamiento, se realizó una prueba de integración utilizando **Thunder Client**.

- **Petición (POST):** Se envió una consulta para identificar qué contratos requieren penalización.
 - **Body:** {"query": "penalizacion_aplicable(X)."}
- **Procesamiento:** El motor analizó los hechos (fechas de entrega, plazos de contratos y estados de cumplimiento).
- **Resultado (JSON):** El sistema identificó correctamente el contrato afectado.

JSON

```
{  
  "status": "success",  
  "results": ["X = c102"]  
}
```

Este resultado confirma que la regla lógica detectó que el contrato c102 superó la fecha límite de entrega.

5. Conclusiones y Extensiones

Conclusiones

- La integración de Prolog en entornos web modernos es viable y poderosa para sistemas de toma de decisiones legales o administrativas.
- El uso de una API REST permite que este motor de lógica sea consumido por cualquier plataforma (móvil, web o escritorio).

Extensiones Futuras

- **Interfaz Gráfica:** Desarrollar un dashboard en React o Vue para visualizar los contratos penalizados en tiempo real.
- **Persistencia de Datos:** Conectar el motor a una base de datos (como MongoDB o PostgreSQL) para que los hechos de Prolog se carguen dinámicamente desde registros reales.

- **NLP:** Integrar procesamiento de lenguaje natural para que el usuario pueda escribir consultas en lenguaje humano y el sistema las traduzca automáticamente a reglas de Prolog.